

AutoTraQ Cheat Sheet — Gus (Backend)

Your Section: Backend (Express + TypeScript + Prisma)

Quick Overview

The backend is a **RESTful API** built with Express.js and TypeScript. It uses **Prisma ORM** to interact with a MySQL database, and follows a clean **Controller** → **Service** → **Repository** architecture.

Tech Stack

- **Express.js** — HTTP server & routing framework
 - **TypeScript** — typed JavaScript for reliability
 - **Prisma ORM** — database toolkit (schema-first, auto-generated client)
 - **MySQL** — relational database
 - **bcrypt** — password hashing
 - **JWT (JSON Web Tokens)** — stateless authentication
 - **Zod** — request validation schemas
 - **Vitest** — test framework
-

Project Structure

```
backend/
├── src/
│   ├── index.ts           ← Server entry point (Express app init)
│   ├── controllers/       ← Handle HTTP requests, call services
│   │   ├── auth.controller.ts ← Login, register, user management
│   │   ├── parts.controller.ts ← CRUD for parts catalog
│   │   ├── inventory.controller.ts ← Stock in/out events
│   │   ├── vehicles.controller.ts ← Vehicle management
│   │   ├── requests.controller.ts ← Part request workflow
│   │   ├── interchange.controller.ts ← Part compatibility
│   │   ├── csv.controller.ts ← CSV import/export
│   │   ├── images.controller.ts ← Part image uploads
│   │   ├── audit.controller.ts ← Audit trail
│   │   ├── notifications.controller.ts ← In-app notifications
│   │   └── partsSearch.controller.ts ← Advanced search
│   └── services/          ← Business logic layer
```

├─ auth.service.ts	← JWT generation, password hashing
├─ parts.service.ts	← Part CRUD logic
├─ inventory.service.ts	← Stock management logic
├─ vehicles.service.ts	← Vehicle logic
├─ requests.service.ts	← Request approval workflow
├─ interchange.service.ts	← Compatibility lookups
├─ barcode.service.ts	← Barcode generation
├─ images.service.ts	← Image storage
├─ sku.service.ts	← SKU auto-generation
├─ notifications.service.ts	← Notification logic
├─ partsSearch.service.ts	← Search queries
├─ routes/	← Route definitions (URL → controller mapping)
├─ middleware/	
│ ├─ auth.middleware.ts	← JWT verification, role checks
│ ├─ error.middleware.ts	← Global error handler
│ └─ validation.middleware.ts	← Zod schema validation
├─ schemas/	← Zod validation schemas for request bodies
├─ utils/	
│ └─ response.ts	← Standardized API response helper
├─ scripts/	
│ ├─ seed-admin-accounts.ts	← Team account seeder
│ └─ seed-vehicles.ts	← Vehicle data seeder
├─ prisma/	
│ ├─ schema.prisma	← Database schema (models, relations)
│ ├─ seed-parts.ts	← Parts catalog seeder
│ └─ seed-sku.ts	← SKU data seeder
├─ tests/	
│ ├─ unit/	← Unit tests
│ └─ integration/	← Integration tests (workflow.test.ts)

Architecture Pattern

Request → Route → Middleware (auth + validation) → Controller → Service → Prisma

- **Routes** map URLs to controllers
- **Middleware** runs first (checks JWT token, validates body with Zod)
- **Controllers** parse the request and call services
- **Services** contain business logic
- **Prisma** handles database queries

Key API Endpoints

Method	Endpoint	Description
POST	/api/auth/login	Login, returns JWT
POST	/api/auth/register	Create new user
GET	/api/parts	List all parts
POST	/api/parts	Create a part
GET	/api/inventory	List inventory events
POST	/api/inventory	Stock in/out event
GET	/api/vehicles	List vehicles
GET	/api/requests	List part requests
POST	/api/requests	Submit request
GET	/api/audit	Audit trail
POST	/api/csv/import	CSV bulk import

Auth System

1. User sends email + password to /api/auth/login
2. Backend hashes password with **bcrypt**, compares
3. Returns signed **JWT** with user ID and role
4. Frontend sends JWT in Authorization: Bearer <token> header
5. auth.middleware.ts verifies token on protected routes
6. Role-based access: admin > manager > fulfillment > viewer

How to Run Locally

```
cd backend
npm install
npx prisma generate      # Generate Prisma client from schema
npx prisma db push       # Sync schema to database
npm run dev              # Starts on http://localhost:3002
```

Running Tests

```
cd backend
npm test                # Runs all Vitest tests
```

```
npm run test:unit          # Unit tests only
npm run test:integration    # Integration tests only
```

Talking Points for the Meeting

- "The backend follows a **layered architecture** — controllers handle HTTP, services handle business logic, Prisma handles data access"
- "We used **Prisma ORM** for type-safe database access — the schema is the single source of truth, and the client is auto-generated"
- "Authentication uses **JWT tokens** with **bcrypt** password hashing — industry standard, stateless, scalable"
- "Request validation uses **Zod schemas** — every endpoint validates input before processing, preventing bad data from reaching the database"
- "We have both **unit and integration tests** using **Vitest** — the integration test covers the full request-to-fulfillment workflow"
- "The audit trail tracks every significant action with user, timestamp, and details for accountability"
- "Role-based access control enforces **four permission levels** — admin, manager, fulfillment, and viewer"